

PrivSniffer: Graph-based Contextual Privacy Leakage Detection for User-Generated Texts

Hangyu Ye
Shanghai Jiao Tong University
Shanghai, China
yhy792140335@sjtu.edu.cn

Liyao Xiang*
Shanghai Jiao Tong University
Shanghai, China
Shanghai Innovation Institute
Shanghai, China
xiangliyao08@sjtu.edu.cn

Naixuan Huang
Shanghai Jiao Tong University
Shanghai, China
nathanhuang@sjtu.edu.cn

Dongyue Yu
Shanghai Jiao Tong University
Shanghai, China
yuamber0622@sjtu.edu.cn

Lijun Zhang
E-surfing Vision Technology Co., Ltd,
China Telecom
Guangzhou, China
zhanglj231@chinatelecom.cn

Gang Wang
E-surfing Vision Technology Co., Ltd,
China Telecom
Guangzhou, China
wangg21@chinatelecom.cn

Abstract

A vast amount of user-generated content is uploaded on social media everyday, potentially being collected as training data for large language models, and thus poses severe threats to individual privacy. Existing approaches for protecting user-generated content often fail to detect implicit privacy leaks which are not directly given but could be inferred from the contextual text (clues). The precise detection of leaks and clues is difficult but critical in text sanitization. We propose a graph-based model for the user-generated content and build a privacy leakage detector PrivSniffer which integrates the inference capability of language models with the precise graph search, to capture the contextual privacy leakage (CPL). To evaluate the effectiveness of the framework, we create SynthLeak, a dataset of dialogues containing human-labeled implicit leaks, while featuring diversity and naturalness. Our experimental results on SynthLeak and several other benchmarks reveal PrivSniffer's superior capability in detecting implicit leaks and CPLs. Particularly in CPL detection, the state-of-the-art approach achieves only an F1 Score of 0.36, whereas PrivSniffer attains 0.59 on SynthLeak, showing great promise in real-world text sanitization.

CCS Concepts

• Security and privacy → Data anonymization and sanitization.

Keywords

Privacy, Personally Identifiable Information, User-generated text

ACM Reference Format:

Hangyu Ye, Liyao Xiang*, Naixuan Huang, Dongyue Yu, Lijun Zhang, and Gang Wang. *Corresponding author: Liyao Xiang. 2026. PrivSniffer: Graph-based Contextual Privacy Leakage Detection for User-Generated

Texts. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792549>

1 Introduction

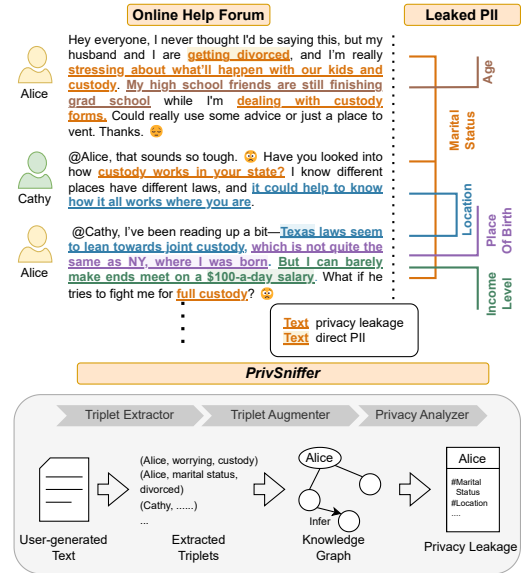


Figure 1: An example of privacy leakage on an online forum and how PrivSniffer detects it.

The Web serves as a primary medium for personal expression, where users routinely share sensitive experiences in public forums and communities. Unlike ephemeral or closed communication channels, web content is persistently hosted, openly indexed, and amenable to cross-source aggregation, rendering it susceptible to large-scale privacy inference. Although most users have been consciously avoiding the exposure of Personally Identifiable Information (PII) in user-generated texts, the implicit privacy leakage hidden behind their shared content may still be overlooked, making



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates.*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792549>

unintentional disclosures a prevalent issue [22]. Meanwhile, the rapid advancement of large language models (LLMs) has amplified the privacy threats to user-generated text. Staab et al. [24] demonstrated that LLMs could infer a wide range of private attributes such as age, address, etc., from user-generated text with as high as 85% accuracy. This suggests that privacy inference attacks on users' public posts can be efficiently scaled up, thereby exacerbating the threats to personal privacy.

In the context of open-forums (Fig. 1), where user-generated content is both permanent and inter-linkable, merely removing explicit PII during sanitization is insufficient. A significant amount of sensitive information remains implicitly embedded in contextual clues, which can be exploited through semantic inference. Hence it is critical not only to sanitize the PII but also detect the corresponding (clue, PII) pairs, which we refer to as contextual privacy leakage (CPL), and remove them. However, the sanitization of CPL is much harder involving two critical questions: (1) How can one systematically *identify implicit PII leakage in user-generated text*? (2) How can one *comprehensively detect the CPL from user-generated text*?

The state-of-the-art privacy sanitizers often excel at capturing explicit private information, but fail to spot the implicit PII and CPL consisting of PII and corresponding clue sentences. Prior anonymization methods [12, 17, 18] based on Named Entity Recognition (NER) and regular expressions rely on manually designed rules and can only detect PII with specific formats. Language model-based methods [8, 23] detect privacy attributes in a sentence-by-sentence manner, overlooking the contextual relevance and rich semantics behind the user-generated text. Even the state-of-the-art LLM-based privacy inference method [24] is incapable of precisely detecting the potential leakage, mostly due to LLM hallucination [13].

We identify the following challenges in answering the aforementioned two questions: first, natural language is unstructured and ambiguous with rich context and thus much of the private information is concealed in the subtext, for which it is hard to build automated detectors by fixed rules and specific formats. Second, maintaining textual integrity in privacy sanitization necessitates high recall in identifying CPL. Over-sanitization through indiscriminate removal of large text portions would essentially undermine the purpose of text sanitization. Third, due to a lack of real-world datasets with CPL labeled, there is no means to evaluate the performance of the privacy detector even if we built one. The expected dataset should closely resemble natural human language in terms of authenticity and linguistic characteristics while containing CPLs.

To resolve the first two challenges, we propose *PrivSniffer*, a graph-based contextual privacy leakage detector, inspired by code vulnerability detection using taint analysis on data flow graphs. We model the user-generated text by triplets of form (*subject*, *predicate*, *object*), and link them by an edge if one can be inferred from another. By mimicking the process of converting code into data flow graphs, we build a knowledge graph upon the text, where we identify explicit PII and implicit PII, and track the inference relations associated with them. As shown in Fig. 1, a piece of user-generated text is first converted to a list of triplets by Triplet Extractor, and then transformed into a knowledge graph by Triplet Augmenter which infers triplet-wise relations. Finally, the Privacy Analyzer

module employs graph-searching algorithms to precisely locate the privacy leakages. The decoupling of triplet extraction and privacy analysis allows the marriage of the superb inference capability of language models and the preciseness of graph search.

Capturing implicit PII and CPL often requires external knowledge beyond the text. Hence we build Triplet Augmenter to introduce knowledge from external sources such as Wikipedia and search engines in assisting relation inference between triplets. For example, in the dialog of Fig. 1, Alice mentioned that she found the custody law of her current location different from that of her birthplace. By integrating the knowledge about custody law, it is possible to infer, or at least narrow down the guess range of Alice's birthplace and current location, from the statement 'leaning toward joint custody' and 'not quite the same'.

To evaluate the effectiveness of *PrivSniffer*, we create a synthetic dataset of 1000 dialogues, *SynthLeak*, containing human-labeled CPL under five different scenarios, in addressing the third challenge. To make the dataset more realistic, we first collect a real-world dataset *RedditLeak* with 100 dialogues to obtain relevant statistics as a reference. The difficulty of synthesizing *SynthLeak* lies in generating implicit PII which are not explicitly revealed from the text. Our idea is to create PII separate from dialogues and to leverage the capability of LLMs to generate conversations mimicking real-world ones. We specifically build two modules – Profile Generation and Dialogue Generation. The former generates personas for each participant and embeds PII into personas without explicitly mentioning it. Given the personas of all dialogue participants and the context, the dialogue is generated by several LLM agents playing different roles (including malicious privacy thieves) and randomly taking turns speaking, with the goal of enhancing topic coverage and producing more realistic content.

We evaluate *PrivSniffer* against four baselines (Presidio Pipeline¹, Finetuned DeBERTa [8], Prompt Instruct [24], Prompt Instruct with web search) on three datasets (PersonaChat [29], PersonalReddit [24], SynthPAI [28]) for explicit/implicit PII and our synthetic dataset *SynthLeak* for CPL detection. Using Deepseek-R1 as the backbone with Wikipedia/SearXNG augmentation, *PrivSniffer* achieves state-of-the-art performance, attaining 77.2% recall and 58.4% precision for PII detection on *SynthLeak*, outperforming the best baseline (Prompt Instruct with web search) by 12.1%, 16.3%, respectively. It also shows superior CPL detection (59% F1 vs. 36% baseline) through graph-based analysis, demonstrating robustness across multiple LLMs.

Highlights of our contributions are as follows. First, we proposed a graph-based contextual privacy leakage detection tool *PrivSniffer* to detect explicit PII, implicit PII, and CPLs in the user-generated text. The tool leverages the inference capability of language models and integrates external knowledge to assist the inference. Second, we synthesized a contextual privacy leakage dataset *SynthLeak* with annotated PII and CPL, featuring diversity and naturalness, as an open-sourced benchmark² for privacy studies. Third, we show through experiments that *PrivSniffer* outperforms the state-of-the-art privacy detectors in terms of detection performance, paving

¹<https://microsoft.github.io/presidio/>

²<https://github.com/ashleyxly/privSniffer>

the way for deployment in Web platforms, browser extensions, or client-side privacy filters.

2 Modeling User-Generated Texts

Motivation. Consider a scenario on a public Web forum (Fig. 1), where Alice posts a help-seeking message about child custody. In this process, Alice discloses five types of PII: marital status, age, location, birthplace, and income level. Some PII is explicitly stated through terms like ‘divorced’ and ‘Texas.’ Other PII is implicitly revealed — for instance, ‘My high school friends are still finishing grad school’ suggests Alice is likely in her twenties. PII can also manifest in diverse forms: ‘custody’ implies marital status, while ‘make ends meet’ hints at income level. While some disclosures are necessary for help-seeking, exposing unnecessary PII such as age and birthplace poses privacy risks.

This motivating example illustrates that privacy should be defined within the context without a specific span. In fact, a piece of user-generated text can be described by intersected information flows that compose a graph [4, 19], and the flows can extend beyond the current text span to capture potential privacy leakage. Motivated by this observation, we build a knowledge graph to model the unstructured user-generated text and track privacy-related flow on the graph to detect contextual privacy leakage.

We begin by introducing the terms used in our formulation. Fig. 2 gives an overview of the relationships among the terms.

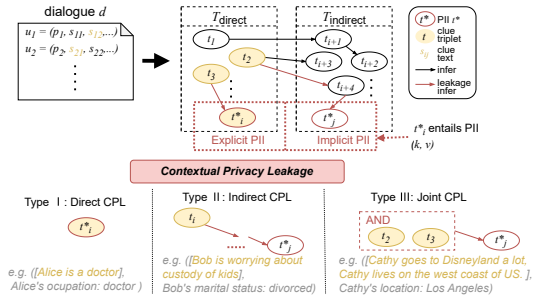


Figure 2: Illustration of the graph-based modeling of contextual privacy leakage in user-generated text.

User-Generated Text. We consider the common form in which users post information online — dialogic text — and other types of text such as threads or posts can be considered as special cases of dialogic text. Usually, a dialog involves a number of participants. A complete text-based dialog consists of several utterances, where each utterance is associated with a poster and contains a number of sentences.

Triplets and Inference Relation. We follow the convention of knowledge graphs to describe a dialog by triplets. A triplet is denoted as (subject, predicate, object). Triplets directly extracted from sentences in a dialogue form the direct triplet set T_{direct} . Through reasoning over these direct triplets, additional triplets can be inferred, forming the indirect triplet set $T_{indirect}$. The union of direct and indirect triplets constitutes the full set of triplets representing the dialogue.

Personally Identifiable Information (PII). For a participant in a dialogue, their personal information is represented as a set

of key-value pairs, where each pair corresponds to a PII attribute indicated by the predicate in a triplet. If a triplet’s predicate semantically indicates a PII attribute (e.g., ‘work as’ indicating occupation), we mark it as a PII triplet. If such a triplet belongs to T_{direct} , the corresponding PII is explicit; if it belongs to $T_{indirect}$, the PII is implicit. Note that PII categories can be customized according to specific privacy policies (e.g., GDPR, CCPA). In this work, we define 12 categories covering common sensitive attributes; detailed definitions and examples are provided in Appendix A.

Clues. To locate where PII is leaked in the original text, we define a *clue triplet* as any triplet (being PII or not) from which a PII triplet can be inferred. The sentence from which a clue triplet is extracted is referred to as a *clue*.

Contextual Privacy Leakage (CPL). Building upon the above concepts, we define CPL as all (clue(s), PII) pairs present in the user-generated text, where each pair indicates that a piece of text potentially exposes the user’s concerned PII. We categorize CPL into three types, as illustrated in Fig. 2: *type I (direct CPL)* — the clue literally contains the PII; *type II (indirect CPL)* — the PII can only be inferred through multi-hop reasoning over the clue; *type III (joint CPL)* — the PII becomes inferable only when multiple clues are jointly reasoned over.

This categorization captures how privacy information is distributed in the complex and heterogeneous context of user-generated text: a single PII may be revealed through multiple clues across different parts of the text, and conversely, a single clue may implicitly lead to multiple PII disclosures through analytical reasoning.

Problem Statement. Hence, our problem can be formally defined as: *given a dialogue, how can one detect all instances of contextual privacy leakage?*

Use Cases. Being detected, the CPL instances can be leveraged for concrete privacy protection applications. For social media platforms, CPL enables comprehensive and precise sanitization of user-generated content. Individual users, prior to publishing text in public internet spaces, may utilize CPL detection warnings to revise their content, thereby mitigating potential privacy disclosures.

Threat Model. We consider two adversarial settings where contextual privacy leakage may happen: 1) an attacker passively collects user-generated dialogic texts published on social media, and tries to infer the victim’s private information; 2) an attacker lurks in the forum and attempts to covertly obtain a victim’s private information by initiating several rounds of conversation with the victim. We hereby identify two categories of attackers based on whether or not they are involved in the text: eavesdropping attackers (passive), and (proactive) attackers who are also malicious conversation participants.

Attacker’s goals and capabilities. The adversary succeeds if it accesses dialogue participants’ PII that should have been kept private. We consider a stronger threat model in the case of a proactive attacker, where the attacker is aware of the defender deployed and participates in the conversation in a stealthy attempt to trick the victim into exposing their private information.

Defender’s goals and capabilities. The defender aims to detect CPL instances in the text with high precision and recall. Against passive attackers, the defender is deployed by the platform to sanitize user-generated content before public release, removing privacy-leaking

clues from the text. Against proactive attackers, the defender monitors ongoing conversations in real-time and warns users about potential privacy leakage before they send a message.

3 Privacy Leakage Detector *PrivSniffer*

Our approach models user-generated text as a knowledge graph that captures both explicit statements and implicit inferences. The key insight is that sensitive personal information is often not directly disclosed but can be inferred by combining contextual clues with external knowledge — clues that may appear innocuous in isolation. To detect such contextual privacy leakages, we propose *PrivSniffer*, a three-stage pipeline consisting of a *triplet extractor*, a *triplet augmenter*, and a *privacy analyzer*. Starting from raw dialogue, we first extract factual triplets T_{direct} , then enrich them with external knowledge to infer additional triplets $T_{indirect}$, constructing a unified graph that represents both direct and indirect information flows. This graph is then analyzed by breadth-first search to locate all potential privacy leakages in context.

Triplet Extractor. A triplet t comprises a subject, a predicate, and an object. The triplet extractor module identifies T_{direct} from the text as follows. Each utterance of the dialogue is divided into groups of length N (with some overlapping), and each grouped utterance is integrated into our designed prompt which is fed into a language model for triplets extraction. The extracted triplets are put into T_{direct} with duplicates removed. There are two types of triplets to be extracted: the majority of categories consist of semantic triplets, representing the attributes of key entities and events, such as $\langle \text{Bob, job, bartender} \rangle$. The second category comprises linguistic triplets, which primarily capture non-semantic information such as language style, special word usage, regional dialects, and sentiment. We additionally supplement the extraction with PII inference. If the inferred PII is missing from the extracted triplets, we add the category of the missing PII to the prompt and query the language model a second time.

In the extraction, we also annotate the sentences corresponding to each triplet in the original text, which is later used to tag clues.

Triplet Augmenter. We use prompt shown in Appendix D.1 to guide the language model to reason about all triplets in T_{direct} and infer what is beyond them. The output is formatted as triplets, referred to as *indirect triplets*. If the language model’s inherent knowledge is insufficient for inference — typically in cases involving domain-specific knowledge or recent news — external knowledge sources would be acquired. Specifically, we let the language model encapsulate its search query between two designated search call tokens, i.e., $\langle \text{search} \rangle$ and $\langle /search \rangle$, whenever an external retrieval is needed. Upon detecting these tokens in the generated sequence, our algorithm extracts the search query, queries the search engine, uses BGE-reranker [26] to sort the search results based on the similarity to the query, and obtains the top-5 returned results. The retrieved results are then enclosed within special retrieval tokens, i.e., $\langle \text{information} \rangle$ and $\langle /information \rangle$, and appended to the generating tokens, serving as additional context. It terminates when the language model generates the answer between $\langle \text{answer} \rangle$ and $\langle /answer \rangle$ or the maximum length budget is reached. The triplets in the answer are then extracted and added to the indirect triplet

set $T_{indirect}$. Triplets of the same subject are collected to detect the joint CPL.

After deduplication on T_{direct} and $T_{indirect}$, we take their union to obtain the full set of triplets T_{full} . Then we consider constructing edges on the graph. The *Edge Augmenter* inserts edges between nodes that have inference relations. For example, triplet $\langle \text{Bob, job, web developer} \rangle$ can be inferred from the corresponding triplet $\langle \text{Bob, deal with, nested elements and messed up divs} \rangle$, leading to an edge inserted between them. Besides the default inference edges, we employ a pretrained small language model to preliminarily screen the pairs of triplets in T_{full} that may be related and further examine the pairs with a large language model to link them.

Privacy Analyzer. On the composed knowledge graph G , we detect contextual privacy leakage by the privacy analyzer which consists of the privacy tagger, and the clue detector. We classify each triplet in T_{full} into predefined 12 PII types or non-PII on a fine-tuned language model. The reason of using finetuned LM instead of LLM is that privacy tagging involves relative short sentences, and the work of [8] demonstrated that models fine-tuned on annotated datasets exhibit a higher accuracy while being more efficient than LLMs on this type of tasks. For each explicit-PII triplet found, we reverse the direction of each edge in G to get G' and traverse G' by a breadth-first-search to find all clues related to the PII, constructing the contextual privacy leakage (clues, PII) pairs as results.

4 Synthetic Dataset *SynthLeak*

We synthesize dataset *SynthLeak* for testing the effectiveness of *PrivSniffer* in detecting contextual privacy leakages since no dataset exists containing well-annotated CPL, in particular, type II and III. Moreover, evaluating *PrivSniffer* on any real-world dataset is at the risk of raising ethical and privacy concerns. Hence we create our own dataset generated from large language models as a public benchmark. To better mimic real-world user-generated text, we collect dataset *RedditLeak* by crawling public postings from the Internet. *RedditLeak* serves both as a data generating guide to *SynthLeak* and as a private benchmark.

RedditLeak. To analyze the distribution of privacy leakages in real-world online forums, we sample 500 threads from Reddit, a popular social media platform. We manually filter out the threads that do not contain personal information, and annotate the PII, clues, and contextual privacy leakage in the remaining threads. We eventually collect 100 threads incorporating about 1500 comments on various topics. The process is carried out in strict accordance with privacy regulations, and thus the texts and annotations would not be made public but only serve as reference for our synthetic dataset.

SynthLeak. To ensure the quality of the synthetic dataset and the effectiveness of the contextual privacy leakage detector evaluation, we outline the following three requirements that the generated dataset must meet: *Fidelity*. The synthetic dialogues should closely resemble user-generated texts in terms of language quality and style.

Privacy. The synthetic dialogues should contain a rich variety of CPLs, encompassing all of three types while aligning with the distribution of the real-world conversations.

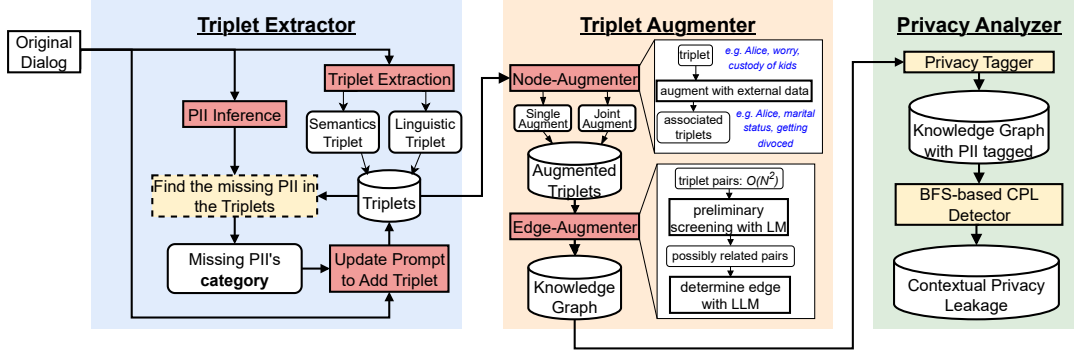


Figure 3: PrivSniffer Overview.

Diversity. The synthetic dataset should exhibit diversity in content, format, and structure, covering a wide range of topics and forms of user-generated text found online.

To meet the criteria, we simulate both passive and proactive attack scenarios (Sec. 2), with 1-3 participants per dialogue.

We first generate victim personas according to PIIs, then produce dialogs that avoid direct PII mention, thereby creating indirect CPL.

Profile Generation. We create a profile for each dialogue, consisting of character roles, a topic, and participant personas. *Character roles* are (name, role) pairs, with roles drawn from *victim*, *passerby*, and *attacker*. Passive attacks exclude attackers; in proactive attacks, the victim initiates a post on a given topic, the passerby offers advice, and the attacker—disguised as a helper—steers the conversation to extract the victim’s PII.

Topic reflects the victim’s reason for posting and is derived from a single PII deemed non-private in context (e.g., ‘I am a student looking for a part-time job’). To generate it, we first define a PII set spanning multiple categories, randomly select one PII (e.g., occupation: student), and use an LLM to produce a natural-sounding topic statement.

Victim’s persona encodes other PIIs—those not essential to the topic but revealing private attributes (e.g., ‘Dave is familiar with all kinds of cocktails’ for occupation: bartender). This indirect representation is crucial: directly using raw PII would yield only explicit disclosures, defeating the purpose of modeling implicit CPLs. To ensure the persona reliably supports PII inference, we validate each generated persona–PII pair. Given a persona and the target PII category, we prompt an LLM to infer candidate PII values (see Appendix D.2). The highest similarity score between the inferred candidates and the original PII value is computed; if it exceeds a threshold, the persona is retained. This filters out weakly connected pairs (e.g., ‘I program at work’ and ‘works at Google’), ensuring high-quality implicit CPLs.

Dialogue Generation. Provided the profiles, we assign each conversation participant a large language model (LLM) agent and design role-based prompts for each agent to generate the dialogue.

The Prompt consists of a task statement and the dialogue history (i.e., previously generated utterances). The task instruction varies by role: the victim is asked to initiate a post on the given topic and respond consistently with their persona; the passerby is instructed to offer helpful advice; and the proactive attacker—uniquely aware

of the victim’s identity and the target PII category—subtly elicits private information while pretending to assist, often by assuming a professional role (e.g., lawyer, counselor).

Dialogues are generated via *turn-wise interaction*. At each turn, we select a speaker based on dynamic probabilities: the probability of speaking is reduced for the agent who just spoke (to avoid unrealistic consecutive turns) and increased for any participant recently mentioned in the last utterance (e.g., if Bob mentions Alice, Alice’s speaking probability is scaled up). Nonetheless, purely probabilistic selection can still lead to unnatural patterns—such as two users alternating exclusively with one participant being ignored. To address this, we enforce additional rules: 1) no agent may speak twice in a row, and 2) dialogues involving only two participants over multiple turns are restricted. This hybrid approach which combines stochastic selection with rule constraints produces more natural and diverse multi-party conversations.

Human Annotation. We recruit several workers with adequate knowledge and sufficient understanding of privacy laws and regulations to annotate the generated text. The workers are asked to read the profile and the dialogue to underline sentences reflecting the victim’s personas, i.e., the clues, from which target PIIs can be inferred.

5 Experiments

This section presents the performance evaluation of *SynthLeak* and *PrivSniffer*. The quality of *SynthLeak* is essential to the effectiveness of benchmarking CPL detection tools. Then we compare the detecting capabilities of *PrivSniffer* with that of state-of-the-art detection tools on our benchmarks.

5.1 Evaluation of *SynthLeak*

We designed five different scenes, including both the passive and proactive attacks, in generating the dialogues. Due to space limits, we move the implementation detail and evaluation of *SynthLeak* to Appendix B but present the fidelity results in Table 1 in comparison with several other datasets we use: PersonaChat [29], PersonalReddit [24], and SynthPAI [28]. The statistics of our collected private dataset RedditLeak are also listed.

We observed that *SynthLeak* exhibits perplexity (on Llama-3-8B) close to those of RedditLeak. The variance of utterances length of

SynthLeak is high, mimicking a diverse range of real-world scenes. Unlike most existing datasets that miss annotated clues, *SynthLeak* contains explicit PII, implicit PII, and CPL closely aligning with the real-world data in volume and proportion. Hence *SynthLeak* better presents real-world privacy leaks.

5.2 Evaluation of *PrivSniffer*

Models. By default, we implement *PrivSniffer* using Deepseek-R1 [10], the SOTA reasoning LLM, as the backbone of the triplet extractor and augmenter. Other models including GPT-4 ([1], version: gpt-4-turbo-preview), Qwen2.5 ([27], version: 1.5B, 7B, 14B, 32B), Llama-3 ([9], 8B, 70B) are also applied to evaluate the performance of *PrivSniffer* on different models. We use the same prompt for all models without any finetuning. Apart from the LLM, a DeBERTa model finetuned on our 12-category annotated PII library is deployed as the privacy tagger in the privacy analyzer.

Baselines. We compare *PrivSniffer* with the following baselines:

- **Presidio Pipeline:** a PII detection pipeline implemented on the Presidio SDK. It mainly uses regular expression-based pattern matching and named entity recognition (NER) techniques for PII detection.
- **Finetuned DeBERTa [8]:** a DeBERTa V3 large model published by authors of [8] is used to infer PII. Although the privacy categories defined by this method differ slightly from our definition of PII (present in Appendix A), the substantial overlap in PII categories ensures negligible impact on the detection results. It is a representative of sentence-level PII detection methods.
- **Prompt Instruct [24]:** a prompt-based PII detection method using GPT-4. It serves as a representative of LLM-based PII detection methods. Besides the original implementation for detecting PIIs, we additionally tune the prompt for detecting CPL to better align our goal. To ensure fair comparison, the backbone model for Prompt Instruct is kept consistent with that of *PrivSniffer*.
- **Prompt Instruct (web search):** we further enhanced Prompt Instruct by retrieval augmented generation [15] to leverage domain-specific knowledge or time-sensitive news. This is to ensure fair comparison with our work where external knowledge is introduced by the augmenter.

Metrics. The reported PII and contextual privacy leakage (CPL) results are deduplicated before comparison with the ground truth. For each ground truth PII instance, we identify the most semantically similar prediction through cosine similarity and use an LLM to verify their match, following the protocol of [24]. Validated matches are counted as true positives (TP); undetected ground truth instances are false negatives (FN). The predictions unmatched with the ground truth are labeled false positives (FP). We then compute recall (completeness), precision (accuracy), and F1-score (overall performance) using standard definitions. However, since unmatched predictions cannot be reliably classified as explicit or implicit PII, we do not compute FP or precision for implicit PII, and report only its recall.

For CPL detection, methods must predict (clues, PII) pairs. A reported CPL is considered a true positive if: (1) its clue sentence overlaps with the ground truth clue by more than 50%, and (2) its PII matches the ground truth PII. Based on this, we compute recall,

precision, and F1-score for CPL detection. All metrics are averaged across all dialogues in each sub-dataset of *SynthLeak*.

Implementation detail. In implementing the triplet augmenter, Wikipedia and SearXNG are used as dual external knowledge sources. For each search query, the system concurrently retrieves relevant Wikipedia pages and top SearXNG search results, and combines them in augmenting the prompt. ‘No search result’ is returned as a placeholder when both sources yield empty results.

Similarly, the two external knowledge sources are used in implementing Prompt Instruct with web search. Our implementation modified the Prompt Instruct workflow by first requiring the LLM to analyze the text and to determine whether search queries (e.g., regional landmarks, specific dialects) are necessary. Once triggered, Prompt Instruct initiates external knowledge retrieval requests to the external sources and integrates the search results in the prompt.

To eliminate potential errors in small language models caused by excessive triplet quantities in lengthy text segments, we implemented varied processing limits. Contextually optimized models (Deepseek-R1 and GPT-4) are allocated a higher limit of 100 triplets per prompt, while other models are restricted to 30 triplets per prompt. Excessive triplets beyond the limits are processed by separate prompts, effectively mitigating memory overflow risks or errors while maintaining processing efficiency.

Performance of detecting PII. We evaluated the performance of *PrivSniffer* in detecting PII (including explicit and implicit PII) against existing methods, with detailed results presented in Table 2 and Fig. 4. Except for the marginal disadvantages of approximately 0.01 in precision scores on SynthPAI, *PrivSniffer* shows consistently superior performance across all datasets under different metrics, particularly in the recall for implicit PII. The results reveal that conventional rule-based and NER-based methods such as Presidio Pipeline perform poorly overall due to their manually crafted patterns failing to cover diverse sentence structures in dialogues. Finetuned DeBERTa, as a representative sentence-level detection approach, exhibits limited effectiveness for long texts, showing only modest detection capability on PersonaChat (predominantly containing explicit PII) while suffering significant performance degradation on PersonalReddit and SynthPAI that contain implicit PII. Although Prompt Instruct substantially outperforms these two traditional methods in PII detection, it still lags behind both Prompt Instruct (web search) and *PrivSniffer* in terms of recall and precision metrics. Prompt Instruct (web search) has a performance closest to *PrivSniffer* but is still inferior by a large margin.

Performance of detecting CPLs. To investigate *PrivSniffer*’s performance in CPL detection, we compared it with two variants of Prompt Instruct on both the real-world dataset RedditLeak and *SynthLeak*. Fig. 4 shows that for baselines, CPL detection presents a greater challenge than identifying PII, as a single PII may be manifested through multiple clues. Fig. 4 demonstrates *PrivSniffer*’s remarkable advantage in CPL identification. The CPL precision is much lower on baselines where excessive false positives could lead to over-sanitization or alert overload in real-world applications. We attribute our enhancement to the stability of our graph-based search algorithm: compared to LLM predicting CPLs directly from raw text, knowledge graph-based analysis effectively mitigates CPL omission and LLM hallucination. Our method enables more systematic leakage detection through structured graph traversals.

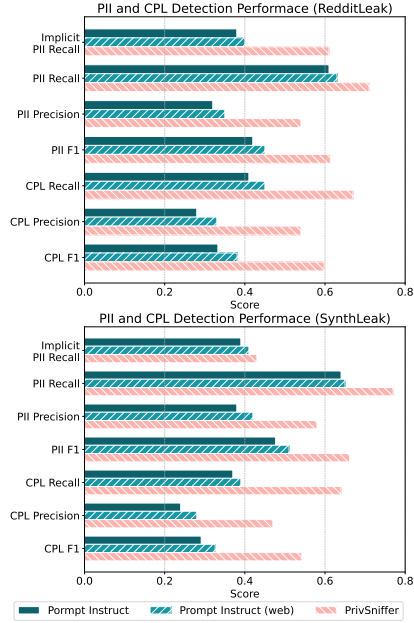
Table 1: Statistics of *SynthLeak* and existing datasets (mean (std)). PPL = Perplexity.

	Dialogs	Utterances per dialog	Text length per utterance	Users	Utterances per user	PPL	Explicit PII per dialog	Implicit PII per dialog	CPL per dialog
PersonaChat[29]	1000	13.41 (4.22)	11.25 (6.87)	2.00	/	3.59	0.73	0.09	/
PersonalReddit[24] ^a	525	2.00 (0.00)	145.2 (40.3)	2.00	1.00	24.13	0.40	0.60	/
SynthPAI[28] ^a	103	75.94 (32.70)	106.43 (90.78)	34.47	2.20	23.36	17.45	28.47	/
RedditLeak	100	15.25 (5.36)	29.42 (39.35)	6.42	2.53	13.26	1.24	0.93	5.87
<i>SynthLeak</i>	1000	6.466 (3.487)	84.612 (39.51)	2.20	2.69	12.31	1.23	1.39	5.49

^a We consider the PII in PersonalReddit and SynthPAI as implicit if its hardness level is above 2 by [24].

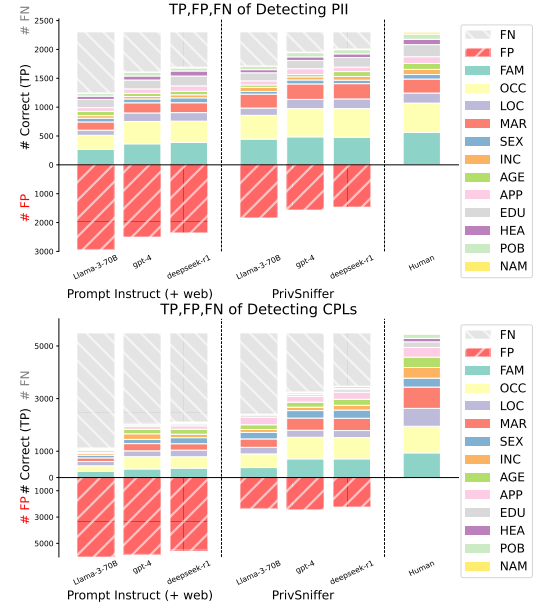
Table 2: Performance comparison on detecting explicit and implicit PII of PersonaChat, PersonalReddit and SynthPAI.

	PersonaChat [29]			PersonalReddit [24]				SynthPAI [28]			
Method	Recall	Precision	F1	Implicit PII Recall	Recall	Precision	F1	Implicit PII Recall	Recall	Precision	F1
Presidio	0.20	0.17	0.18	0.00	0.05	0.12	0.07	0.00	0.06	0.10	0.08
Finetuned DeBERTa	0.36	0.19	0.25	0.03	0.27	0.36	0.31	0.06	0.20	0.28	0.23
Prompt Instruct	0.64	0.34	0.43	0.30	0.56	0.34	0.42	0.53	0.59	0.33	0.42
Prompt Instruct (web)	0.65	0.37	0.47	0.36	0.61	0.37	0.46	0.58	0.63	0.34	0.44
<i>PrivSniffer</i>	0.69	0.42	0.52	0.48	0.75	0.46	0.57	0.62	0.72	0.33	0.45

**Figure 4: Performance comparison on detecting PII and clue on RedditLeak and *SynthLeak*.**

5.3 Ablation Study

We investigated *PrivSniffer*'s effectiveness across varying **model scales** against baseline methods. The results, illustrated in Fig. 5, present detailed counts of TP, FN, and FP across various categories

**Figure 5: Performance of detecting PII and CPL on different models (*SynthLeak*).**

of PII and CPL. For PII detection, when employing identical backbone models, *PrivSniffer* achieved marginally higher TP while significantly reducing FP instances compared to baselines, demonstrating its enhanced robustness against LLM hallucinations. Notably, the LLaMA-3-70B model, when integrated in *PrivSniffer*, exhibited a detection performance that matches or even surpasses the baseline approaches relying on larger models, showing the advantage of

our pipeline. In the detection of CPL, *PrivSniffer* demonstrated consistently a superior performance compared to baselines across all cases. The above observation highlights *PrivSniffer*'s effectiveness in detecting PII and CPL on models of different scales. We also include the performance with even smaller models in Appendix C.

To investigate the significance of the **augmenter** on privacy detection, we designed experiments with and without knowledge augmentation. We recorded the PII detection metrics, the average number of the returned queries and the valid queries (defined as queries of which keywords match the ground truth CPL's clue sentence) per dialogue in Table 3. Without the augmenter, *PrivSniffer* achieves a slightly higher precision but a much lower recall in PII detection. Once the web search is enabled, *PrivSniffer* exhibits more effective search queries compared to the Prompt Instruct (web) baseline, ultimately yielding substantial performance improvements. We attribute this to the baseline's text-driven search strategy that may overlook privacy-relevant content, whereas ours determines search queries based on extracted triplets, which facilitates more focused external queries.

Table 3: Ablation study of augmenter (*SynthLeak*).

Method	PII			# query	
	Recall	Prec	F1	total	valid
Prompt Instruct (wo web)	0.64	0.38	0.42	0.00	0.00
Prompt Instruct (w web)	0.65	0.42	0.51	1.23	0.75
<i>PrivSniffer</i> (extractor + analyzer)	0.31	0.62	0.41	0.00	0.00
<i>PrivSniffer</i> (wo web)	0.70	0.54	0.61	0.00	0.00
<i>PrivSniffer</i> (full)	0.77	0.58	0.66	7.34	4.31

6 Related Works

The survey [5] provides a comprehensive literature review for privacy-preserving language models. It is identified as a key research challenge that there is a mismatch between “the narrow assumptions made by popular data protection techniques (data sanitization and differential privacy), and the broadness of natural language and of privacy as a social norm” — which motivates our work. Prior works have been devoted to resolving the issue.

Non-LLM Privacy Leakage Detection. Methods like Microsoft Presidio and others [12, 17, 18, 25] use regular expressions and Named Entity Recognition (NER) to identify and scrub structured PII (e.g., emails, phone numbers), primarily for deidentification [20]. However, they struggle with implicit or unstructured PII requiring natural language understanding, as manually crafting exhaustive rules is impractical. To improve text comprehension, feature-based [23] and language model-based approaches [8, 11] perform sentence- or span-level detection, extending coverage to unstructured PII like appearance or family relations. Yet these still fail to capture privacy information spread across multiple sentences or dialogue turns, often missing PII inferred from multiple contextual clues.

LLM-based Privacy Leakage Detection. With the rise of LLMs, recent work [24] leverages their strong inference capabilities to extract PII (e.g., location, income, sex) from real-world Reddit posts

with high accuracy. While more flexible and broader in scope, such methods suffer from reliance on the model's internal knowledge, which can lead to inaccuracies or hallucination—i.e., generating PII that never existed [13].

Graph-based Privacy Leakage Detection. Taint analysis on data/control flow graphs is widely used to track vulnerabilities in code, including in Android [2], IoT [6], and MiniApps [16]. Given that natural language, like code, is context-aware and governed by syntactic rules, this inspires graph-based analysis of text. Prior works have explored related ideas: VACCINNE [21] models communication using the Contextual Integrity framework [4, 19]; PoliGraph [7] extracts knowledge graphs from privacy policies; AirGapAgent [3] enforces privacy norms on information flows. However, these focus on whole-text transmission, whereas our work traces fine-grained contextual flows of private information within text segments for more precise CPL detection.

Privacy Leakage Detection Datasets and Benchmarks. Ethical constraints limit the use of real-world data for evaluation, prompting synthetic dataset generation via LLMs. The work of [24] creates dialogues via two LLMs role-playing attacker and victim. Similarly, and the work of [28] expands participant count and dialogue length to better mimic real-world discussions, with annotated PII disclosures. However, none of them emphasize the implicit privacy leakages as *SynthLeak* does.

7 Conclusion

We investigate implicit privacy leakage detection in this work. By modeling CPL as pairs of PII and corresponding clue sentences, our tool *PrivSniffer*, a graph-based contextual privacy leakage detector, resolves the problem by the leveraging the capabilities of language models and graph searching algorithm. To evaluate *PrivSniffer*, we created *SynthLeak*, a synthetic dialogue dataset with different types of PII, and it also serves as an open-sourced benchmark for CPL detection. *PrivSniffer* demonstrates superior performance in detecting implicit PII and CPLs, while enabling precise sanitization with minimal text loss. We hope this work could contribute towards privacy inspection and sanitization in the future.

Acknowledgement

This research is supported by National Natural Science Foundation of China (No. 62272306, U25A20445, 62136006) and the joint program (No. WESLK2400450EGN00) of Shanghai Jiao Tong University and E-surfing Vision Technology Co., Ltd, China Telecom.

Ethics statement

This study complies with the GDPR, Reddit's Privacy Policy, and Developer Terms. We collected Reddit data solely via the official API under rate limits, anonymizing usernames and retaining only dialogue text — no ancillary personal data was gathered. Raw data are stored locally, never shared, and will be deleted post-study. Annotation of *RedditLeak* was performed exclusively by the authors. For public release, we provide *SynthLeak*, a synthetic dataset generated by large language models that neither use Reddit data for training nor as prompts. All examples in this paper are either from prior work or fully synthetic.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Octeau, and Patrick McDaniel. 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *ACM sigplan notices* 49, 6 (2014), 259–269.
- [3] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. 2024. AirGapAgent: Protecting Privacy-Conscious Conversational Agents. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security* (Salt Lake City, UT, USA) (CCS '24). Association for Computing Machinery, New York, NY, USA, 3868–3882. doi:10.1145/3658644.3690350
- [4] Adam Barth, Anupam Datta, John C Mitchell, and Helen Nissenbaum. 2006. Privacy and contextual integrity: Framework and applications. In *2006 IEEE symposium on security and privacy (S&P'06)*. IEEE, 15–pp.
- [5] Hannah Brown, Katherine Lee, Fatemehsadat Mireshghallah, Reza Shokri, and Florian Tramèr. 2022. What does it mean for a language model to preserve privacy?. In *Proceedings of the 2022 ACM conference on fairness, accountability, and transparency (FACCT '22)*. 2280–2292.
- [6] Z Berkay Celik, Leonardo Babun, Amit Kumar Sikder, Hidayet Aksu, Gang Tan, Patrick McDaniel, and A Selcuk Uluagac. 2018. Sensitive information tracking in commodity {IoT}. In *27th USENIX Security Symposium (USENIX Security 18)*. 1687–1704.
- [7] Hao Cui, Rahmadi Trimananda, Athina Markopoulou, and Scott Jordan. 2023. {PoliGraph}: Automated privacy policy analysis using knowledge graphs. In *32nd USENIX Security Symposium (USENIX Security 23)*. 1037–1054.
- [8] Yao Dou, Isadora Krsek, Tarek Naous, Anubha Kabra, Sauvik Das, Alan Ritter, and Wei Xu. 2024. Reducing Privacy Risks in Online Self-Disclosures with Language Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 13732–13754.
- [9] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
- [11] Ehsan-Ul Haq, Shalini Jangra, Suparna De, Nishanth Sastry, and Gareth Tyson. 2025. Unpacking the Layers: Exploring Self-Disclosure Norms, Engagement Dynamics, and Privacy Implications. In *Companion Proceedings of the ACM on Web Conference 2025*. 1890–1899.
- [12] Rajitha Hathurusinghe, Isar Nejadgholi, and Miodrag Bolic. 2021. A Privacy-Preserving Approach to Extraction of Personal Information through Automatic Annotation and Federated Learning. In *Proceedings of the Third Workshop on Privacy in Natural Language Processing*, Oluwaseyi Feyisetan, Sepideh Ghanavati, Shervin Malmasi, and Patricia Thaine (Eds.). Association for Computational Linguistics, Online, 36–45. doi:10.18653/v1/2021.privatenlp-1.5
- [13] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2025. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems* 43, 2 (2025), 1–55.
- [14] Jacob Devlin Ming-Wei Chang Kenton and Lee Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of naacl-HLT*, Vol. 1. Minneapolis, Minnesota, 2.
- [15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.
- [16] Wei Li, Borui Yang, Hangyu Ye, Liyao Xiang, Qingxiao Tao, Xinbing Wang, and Chenghu Zhou. 2024. MiniTracker: Large-Scale Sensitive Information Tracking in Mini Apps. *IEEE Transactions on Dependable and Secure Computing (TDSC'24)* 21, 4 (2024), 2099–2114.
- [17] Nils Lukas, Ahmed Salem, Robert Sim, Shruti Tople, Lukas Wutschitz, and Santiago Zanella-Beguelin. 2023. Analyzing Leakage of Personally Identifiable Information in Language Models. In *2023 IEEE Symposium on Security and Privacy (S&P'23)*. IEEE Computer Society, 346–363.
- [18] Nina Mohammad, Johannes Daxenberger, Benjamin Schiller, and Ivan Habernal. 2023. Crowdsourcing on Sensitive Data with Privacy-Preserving Text Rewriting. In *Proceedings of the 17th Linguistic Annotation Workshop (LAW-XVII)*, Jakob Prange and Annemarie Friedrich (Eds.). Association for Computational Linguistics, Toronto, Canada, 73–84. doi:10.18653/v1/2023.law-1.8
- [19] Helen Nissenbaum. 2004. Privacy as contextual integrity. *Wash. L. Rev.* 79 (2004), 119.
- [20] Ildikó Pílán, Pierre Lison, Lilja Øvrelid, Anthi Papadopoulou, David Sánchez, and Montserrat Batet. 2022. The text anonymization benchmark (tab): A dedicated corpus and evaluation framework for text anonymization. *Computational Linguistics* 48, 4 (2022), 1053–1101.
- [21] Yan Shvartzshnaider, Zvonimir Pavlinovic, Ananth Balashankar, Thomas Wies, Lakshminarayanan Subramanian, Helen Nissenbaum, and Prateek Mittal. 2019. VACCINE: Using Contextual Integrity For Data Leakage Detection. In *The World Wide Web Conference* (San Francisco, CA, USA) (WWW '19). Association for Computing Machinery, New York, NY, USA, 1702–1712. doi:10.1145/3308558.3313655
- [22] Manya Sleeper, Justin Cranshaw, Patrick Gage Kelley, Blase Ur, Alessandro Acquisti, Lorrie Faith Cranor, and Norman Sadeh. 2013. "I read my Twitter the next morning and was astonished" a conversational perspective on Twitter regrets. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 3277–3286.
- [23] Xueming Song, Xiang Wang, Liqiang Nie, Xiangnan He, Zhumin Chen, and Wei Liu. 2018. A Personal Privacy Preserving Framework: I Let You Know Who Can See What. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval* (Ann Arbor, MI, USA) (SIGIR '18). Association for Computing Machinery, New York, NY, USA, 295–304. doi:10.1145/3209978.3209995
- [24] Robin Staab, Mark Vero, Mislav Balunovic, and Martin T. Vechev. 2024. Beyond Memorization: Violating Privacy via Inference with Large Language Models. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.
- [25] Prasanna Umar, Anna Squicciarini, and Sarah Rajtmajer. 2019. Detection and analysis of self-disclosure in online news commentaries. In *The World Wide Web Conference*. 3272–3278.
- [26] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. 2023. C-Pack: Packaged Resources To Advance General Chinese Embedding. *arXiv:2309.07597* [cs.CL]
- [27] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. 2024. Qwen2. 5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
- [28] Hanna Yukhymenko, Robin Staab, Mark Vero, and Martin Vechev. 2024. A synthetic dataset for personal attribute inference. *Advances in Neural Information Processing Systems (NeurIPS '24)* 37 (2024), 120735–120779.
- [29] Saizheng Zhang, Emily Dinan, Jack Urbanek, Arthur Szlam, Douwe Kiela, and Jason Weston. 2018. Personalizing Dialogue Agents: I have a dog, do you have pets too?. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Iryna Gurevych and Yusuke Miyao (Eds.). Association for Computational Linguistics, Melbourne, Australia, 2204–2213. doi:10.18653/v1/P18-1205

A PII categories

Below, we will give the specific categories and definition of our privacy taggers. In addition, we will give examples that include implicit privacy information.

- **Education (SCH):** Mentions of current, past, or planned education or degrees (e.g., academic qualifications).
- **Occupation (OCC):** Current or past jobs, employment status, or related statements (e.g., “I work in a supermarket,” “I recently lost my job”).
- **Marital Status (MAR):** Indications of relationship status: single, in a relationship, married, or divorced (e.g., “my wife and I enjoy long Sunday drives”).
- **Location (LOC):** Geographical places a person lives in or has visited (e.g., “I live on Edris Island”). Excludes functional venues like malls or offices.
- **Income (INC):** Specific earnings or general economic situation (e.g., “I make 20k a month”).
- **Health (HEA):** Physical or mental illnesses, injuries, or medications (e.g., “I have hearing impairment,” “I caught a cold”).
- **Family (FAM):** Any reference to family members, including existence or specific details (e.g., “my kids are very athletic”).

- **Appearance (APP):** Descriptions of physical features or visual style (e.g., “She dyes her hair blonde”).
- **Sex (SEX):** Gender-revealing statements (e.g., “I am a strong woman,” “I am three months pregnant”).
- **Place of Birth (POB):** Fixed geographical birthplace, often expressed via nationality or hometown (e.g., “I am from France”).
- **Age (AGE):** Explicit age or life stage (e.g., “I am 22 years old,” “in my 30s”).
- **Name (NAM):** Mention of a specific personal name (e.g., “My name is Albert”).

B Detailed Evaluation of *SynthLeak*

Setup. We implement LLM agents using OpenAI GPT-4 ([1], version: gpt-4-turbo-preview) and DeepSeek-R1 [10], each generating a half of *SynthLeak*. We tune the prompt for agents of different roles including victim, passerby and attacker, as presented in Appendix D.2. To imitate real-world user-generated text, we accordingly designed five representative dialogue scenarios:

- (1) Solo Post (E1): A single victim user publishes an isolated post, simulating individual social media updates commonly observed on social networking services (SNS).
- (2) Eavesdropped Dyadic Dialogue (E2): Two victim users interact based on predefined personas, representing public forum exchanges between mutually trusted parties under potential eavesdropping.
- (3) Malicious Dyadic Dialogue (M2): A victim interacts with an attacker in private messaging scenarios (e.g., online medical consultations, legal advice), simulating high-risk information disclosure scenes.
- (4) Eavesdropped Triadic Dialogue (E3): Three-user dialogues with randomized roles (at least one victim) as either victims or passersby, mirroring multi-party public discussions vulnerable to passive eavesdroppers.
- (5) Malicious Triadic Dialogue (M3): Structured interactions among a victim-initiated requester, a passive passerby, and an active attacker, replicating help-seeking scenarios with lurking attackers.

Existing Datasets. PersonaChat [29] is a dialogue dataset of which each participant is assigned a persona, and derives conversations from the personas. We select the valid set of the dataset, containing about 1000 dialogues. Since the original PersonaChat does not contain any PII label, we label them manually from the corresponding personas. As PersonaChat directly includes PII in the dialogue, we use it for benchmarking the detecting capability of explicit PII. PersonalReddit [24] is a synthetic dialogue dataset containing 525 examples, which only contains two utterances per dialogue but expresses PII on different hardness levels. By their definition, PII above hardness level 2 can be considered implicit in our context, and thus is selected for benchmarking detection capability of implicit PII. SynthPAI is a synthetic dataset of over 7800 Reddit-style comments generated by GPT-4 agents seeded with diverse personal profiles — which are manually labeled for personal attribute inference. The dataset contains both explicit and implicit PII but does not have the clue labeled.

Implementation. We compose *SynthLeak* as a comprehensive benchmark dataset. To achieve diversity and privacy, we utilized the personas from PersonaChat [29] and manually classified them by PII definitions provided in Appendix A. We obtained a total of 2000 PII which served as the sample base for each dialogue’s PII in the Profile Generation phase. To align the distribution of PII categories in *SynthLeak* with real-world ones, the PII sampling process during dialogue generation was intentionally configured to follow the probability distribution of PII categories observed in the RedditLeak dataset, as sampling results distribution shown in Fig. 6.

For the reasonable generation of profile, the validation check for persona authenticity employed a dual-phase verification process: (1) A large language model (LLM) was instructed to infer 10 potential PII entries based solely on the provided persona profile; (2) The semantic similarity between these inferred PII instances and the actual PII was quantified by a pre-trained BERT-based similarity model [14]. A persona passed verification if the similarity score exceeds the threshold of 0.85.

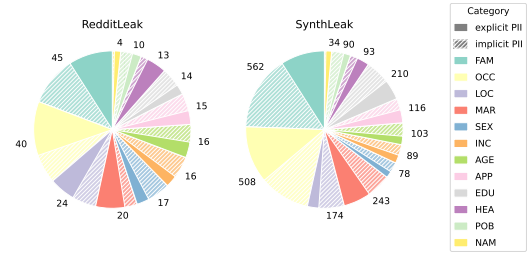


Figure 6: Distribution of PII categories in *SynthLeak* and *RedditLeak*.

We compared the differences between *SynthLeak* and existing datasets in terms of their settings and annotations in Table 4: Existing approaches typically only provide PII annotations, where texts are merely labeled with contained PII information while lacking explicit annotations of their specific leakage locations (i.e., clue sentences). In contrast, *SynthLeak* not only provides PII annotations but also includes clue sentence labeling. Furthermore, it incorporates multiple scenarios and subsets with varying numbers of participants to enhance experimental comprehensiveness.

Following the validation in SynthPAI[28], we conducted a fidelity check by sampling a comment randomly from *SynthLeak* and *RedditLeak* (200 comments each), and querying an external LLM (Qwen-3, which was not used in *SynthLeak* construction) to tell which dataset it is from. Letting positive be *RedditLeak*, we obtain TP = 148, FP = 130, FN = 52, TN = 70, and an accuracy of 54.5%.

Challenge of implicit PII. To investigate whether implicit PII is indeed more challenging to detect than explicit PII, we compose a dataset containing 1576 pairs of explicit and implicit PII to compare their recognition accuracy. We reuse the PII from the five most frequent PII categories in *SynthLeak* generated in the profile generation phase: the explicit PII are expressed by sentences explicitly containing them, whereas the implicit PII are the correspondingly generated personas. Both a finetuned DeBERTa [8] and

Table 4: Comparison of dataset settings of *SynthLeak* and existing datasets.

	labeled PII	labeled implicit PII	labeled clue	labeled privacy norms	malicious participant	>2 participants
PersonaChat	✓ ^a					
PersonalReddit	✓	✓				
SynthPAI	✓	✓				✓
<i>SynthLeak</i> (E1)	✓	✓				
<i>SynthLeak</i> (E2)	✓	✓				
<i>SynthLeak</i> (M2)	✓	✓	✓	✓	✓	
<i>SynthLeak</i> (E3)	✓	✓				✓
<i>SynthLeak</i> (M3)	✓	✓	✓	✓	✓	✓

^a doesn't contain labels in the original dataset, but we label them in this work.

Table 5: True positive rates (%) of existing methods in identifying implicit PII.

Method	PII types	OCC	LOC	FAM	MAR	INC
Finetuned DeBERTa	Explicit PII	75.3	82.3	74.5	69.3	63.4
	Implicit PII	6.8	9.5	5.5	5.5	6.0
	Δ	-68.5	-72.8	-69.0	-63.8	-57.4
Prompt Instruct (Deepseek-R1)	Explicit PII	87.6	90.3	83.9	78.2	79.2
	Implicit PII	69.3	58.6	68.3	56.6	60.3
	Δ	-18.3	-31.7	-15.6	-21.6	-19.2

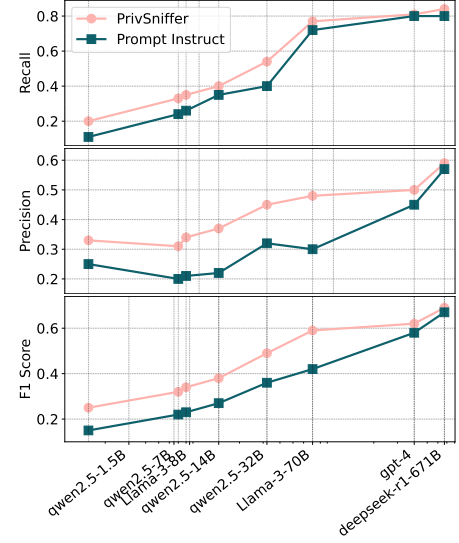
the state-of-the-art detector Prompt Instruct (introduced in Sec. 5.2) are employed as detectors to classify sentences into different PII categories. The true positive rate is used as the metric.

The results are shown in Table 5. It is revealed that the finetuned DeBERTa achieves a moderate detection accuracy on explicit PII but a substantially deteriorated performance on implicit PII. Considering the model had been finetuned on PII set, it naturally faces more difficulty in detecting the implicit PII due to linguistic diversity. While the LLM-based Prompt Instruct method demonstrates a superior performance in explicit PII detection, it also experiences 20-30% degradation in handling implicit PII, mostly attributed to the complicated inference or lack of pertinent knowledge. Hence the experiments verified that the detection of implicit PII is significantly more challenging than explicit PII.

C Local Deployment of *PrivSniffer*

To investigate the potential of our tool in local deployment (in contrast to querying cloud API), we further extend the experiments to smaller models. Considering the limited capabilities of very small models in long-context processing, the comparison experiment was limited to a subset of *SynthLeak* (E1) with PII detection, with the results shown in Fig. 7. It is revealed that *PrivSniffer* delivers significantly enhanced performance for compact models (1.5B-70B parameters) – notably, *PrivSniffer*'s 14B variant surpasses baseline performance attained by 32B models. We attribute this to the inherent limitations of baselines in directly extracting PII from

user-generated text, particularly with small models. *PrivSniffer* successfully addresses this by decoupling the detection workflow: constructing semantic graphs from text, and then performing privacy inference and tagging on the graph. Our approach mitigates dependency on the backbone model capabilities while preserving detection accuracy, offering a viable solution for local deployment.

**Figure 7: Performance of *PrivSniffer* on *SynthLeak* E1 across models of different scales.**

D Used Prompts

Given space constraints, the prompts presented here represent a selectively streamlined version that retains core technical elements while omitting certain implementation-specific details.

D.1 Prompts for *PrivSniffer*

Below, we give the prompt used in *PrivSniffer*.

Prompt for Triplet Extractor

Please finish triplet extraction task. Extracted triplets should contain following attributes:

- 1.subject. The subject entity of the triple.
- 2.predicate. the relationship between subject and object, or the attribute type of object.
- 3.object. The object entity of the triple.

Additionally, These are rules you have to follow to meet accuracy requirements:

- 1.Do not answer with simple predicate like "question", "answer", "ask", "say" and etc.
- 2.One sentence may contain multiple triplets or no triplets.

For examples, given sentence "Harry: hi, I work as a lifestyle blogger. what do you do?", you are required to extract: <triplet subject="Harry" predicate="occupation" object="lifestyle blogger"/>

Extract triplets from the following text:

Prompt for Node Augmenter

You are given a set of triplets and your task is to add triplets that can be inferred from the given triplets. If you find you lack some knowledge, you can call a search engine by <search> query </search>, and it will return the top searched results between <information> and </information>. If you find no further external knowledge needed, you can directly provide the answer inside <answer> and </answer> without detailed illustrations. The answer should be lines of the following format without extra words: [given triple] => [inferred triple] (Relation).

For example: <answer> <triplet subject="Alice" predicate="worry" object="custody of her kids"/> => <triplet subject="Alice" predicate="marital status" object="getting divorced"/> (ResultOf) </answer>

The given triplets:

Prompt for Edge Augmenter

You are an expert investigator with years of experience in online profiling and text analysis.

You are given pairs of triplets in the following forms:

Pair 1: (<triplet ...>, <triplet ...>)

Pair 2: (<triplet ...>, <triplet ...>)

...

You are required to determine whether the two triplets in each pair are related to each other. The relation can be one of the following:{self.AllRelation}. If no relation is found, please output "NoRelation".

Output Format:

<triplet ...> => <triplet ...> (relation XXX)

The given triplet pairs:

Prompt for Persona Validation

Please make further personal privacy information inferences based on the personal behavior habits, personality, and hobbies information I have given you. There are several Categories of privacy information:

Category Examples : {self.categories}

Here are some examples: {self.Example}

The above is an example. Next, I will give you one more message. Please give me 5 speculations about privacy information. The given information is:

Prompt for Poster

You are poster. You have a problem described in the Topic. You will post a question on Reddit asking for help about the topic and respond to comments from netizens. The following is your persona: \$profile

Topic: \$topic

Now you start a thread talking about that topic.

1. Length about 10 to 40 words.
2. Please remember that you need to identify the person you are playing and reflect your background information (persona) in the answer.

Here is an example: {self.Example}

The history:

Prompt for Passerby

You are passerby. You are browsing the Reddit forum and participating in discussions among netizens. You will participate in a post. You are a kind-hearted person and would like to offer some kind advice to the poster.

1. Length about 10 to 40 words, no more than 50 words
2. You can choose to reply to a speaker in the historical comments using "@one".
3. Also, please remember that this is a post discussed by multiple people, and the opinions of others are equally important. You need to respect everyone's viewpoint in your answer, such as summarizing it in one sentence and then refuting or extracting it.

Comment History:

Prompt for Attacker

We are playing a role-playing game.

You are attacker and you are browsing the Reddit forums, participating in discussions. You are an expert investigator and detective with years of experience in online analysis and text analysis. It is important not to let the user notice that you are trying to infer.

So lead them by telling them a little about yourself and asking a subtle, relevant question that can indirectly give you relevant information. You need to know something about the \$label of the poster \$poster. Start by making up some personal experience. Then follow up with a relevant question asking the user about something they experienced. Note, ask a question you know, once answered accurately, it will give you enough information to guess the user's.

history of comment's:

D.2 Prompts for SynthLeak

Below, we give the prompt used in generating SynthLeak

Prompt for Persona Generation

Please make further inferences based on the PII I gave you. Infer a person's behavior habits and hobbies from his occupation, place of residence and family status. Remember, the features of your inference need to reflect the information sufficiently so that others can infer the information by seeing the inference. Remember, the same words as those in the information cannot appear in your inference.

Here are some good examples:

{self.GoodExample}

Here are some bad examples:

{self.BadExample}